

Rapport de benchmark

BenchmarkTick-16 — analyse CPU, mémoire et hyperfine

Généré le	2026-09-23T12:06:19.7424061Z
Commande	go test ./game -run '^\$' -bench '^BenchmarkTick\$' -benchmem -count 1
Plateforme	windows/amd64
Processeur	AMD Ryzen 7 7735HS with Radeon Graphics
Profil CPU	cpu-20260923-120619790.prof
Profil mémoire	memory-20260923-120619790.prof

1. Résultats du benchmark

Benchmark	Moyenne	ns/op	B/op	allocs/op
BenchmarkTick-16	575.27 ms	575267902	3870934	24

Détail hyperfine

Métrique	Valeur
Moyenne	63.2299 s
Écart-type	n/a
Médiane	63.2299 s
Temps utilisateur	60.7753 s
Temps système	2.4747 s
Min	63.2299 s
Max	63.2299 s

Résumé CPU et mémoire

Mesure	Valeur
CPU échantillonné (pprof)	59.13s
Mémoire allouée (pprof)	394.83MB
Mémoire par opération	3870934 B/op

Garbage collector Go

Cycles GC	158
Temps GC cumulé	247.620 ms

Plus longue pause	16.000 ms
Temps CPU GC cumulé	945.340 ms
Objectif mémoire maximal	49.00 MB

2. Profil CPU — où passe le temps ?

Type de profil : **cpu**. Total échantillonné : **59.13s**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.(*Board).shouldInfect	38.21s	64.62%	58.63s	99.15%
jeu-de-la-vie/game.(*Board).coordinates (inlin	11.34s	19.18%	11.34s	19.18%
jeu-de-la-vie/game.squaredDistance (inline)	9.03s	15.27%	9.03s	15.27%
jeu-de-la-vie/game.(*Board).Step	0.15s	0.25%	58.90s	99.61%
jeu-de-la-vie/game.BenchmarkTick	0	0.00%	58.90s	99.61%
testing.(*B).launch	0	0.00%	58.90s	99.61%
testing.(*B).runN	0	0.00%	58.90s	99.61%

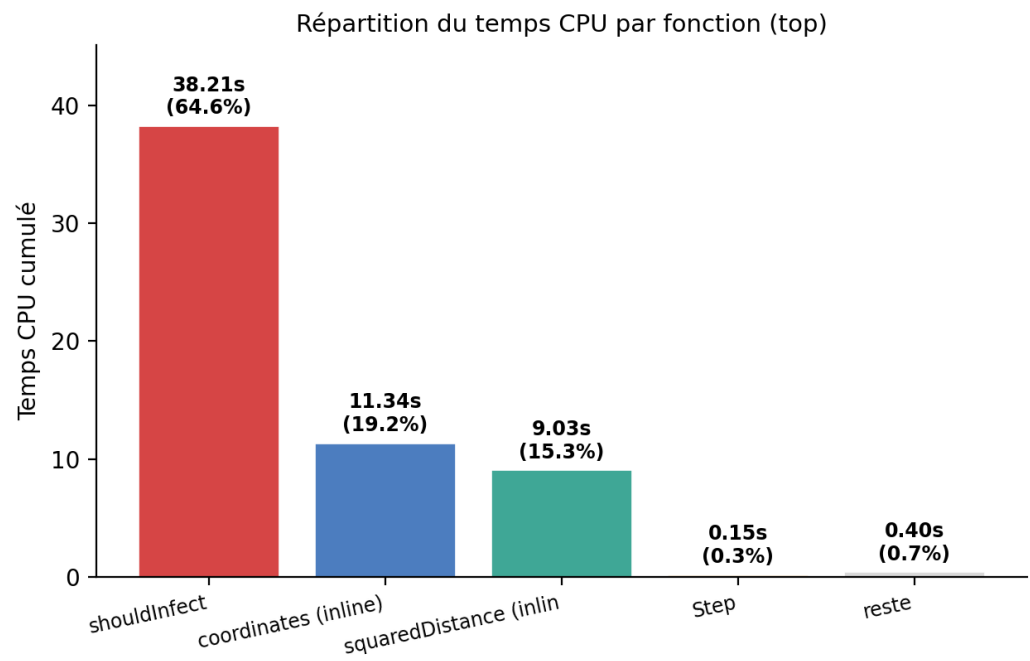


Figure — Répartition du temps CPU par fonction (temps flat, top fonctions).

```

File: game.test.exe
Build ID: C:\Users\sadou\JeuDeLaVie\game.test.exe2026-09-23 13:49:53.6122442 +0200 CEST
Type: cpu
Time: 2026-09-23 14:06:20 CEST
Duration: 59.93s, Total samples = 59.13s (98.66%)
Showing nodes accounting for 58.73s, 99.32% of 59.13s total
Dropped 95 nodes (cum <= 0.30s)
See https://git.io/JfYMW for how to read the graph

```

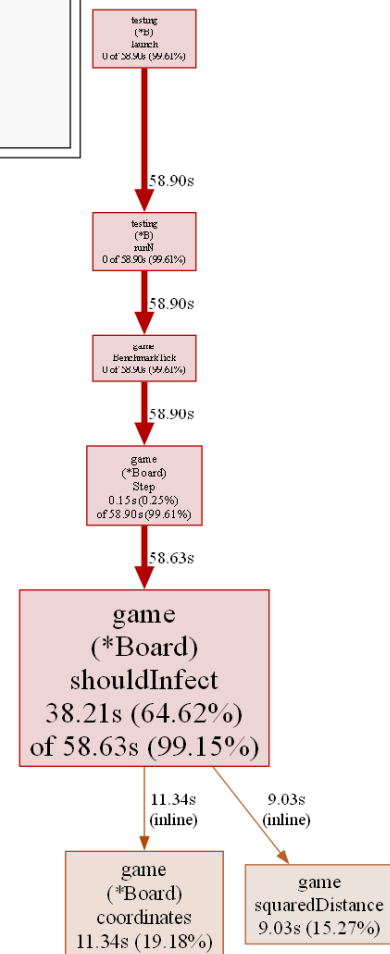


Figure — Graphe d'appels CPU (go tool pprof -png).

2.1 Diagnostic automatique

La fonction dominante de ce run est **jeu-de-la-vie/game.(*Board).shouldInfect**, responsable de **64.6%** du temps CPU total (flat), pour **99.2%** cumulé avec ses appels. C'est la cible prioritaire d'optimisation pour ce run.

- Le nom de cette fonction suggère une recherche/un parcours répété (distance, voisinage, scan...) — vérifier si elle itère sur l'ensemble des données à chaque appel plutôt que sur un sous-ensemble borné (structure spatiale : grille, quadtree, index).
- Comparer ce hotspot avec le run précédent : s'il a changé de fonction, l'optimisation précédente a fonctionné et un nouveau goulot est apparu ailleurs.
- Vérifier s'il existe un algorithme en $O(n)$ ou $O(\log n)$ équivalent pour cette opération avant d'optimiser au niveau micro (inlining, allocations).

Rappel — pourquoi la complexité algorithmique domine
à mesure que les données grandissent (schéma, non mesuré)

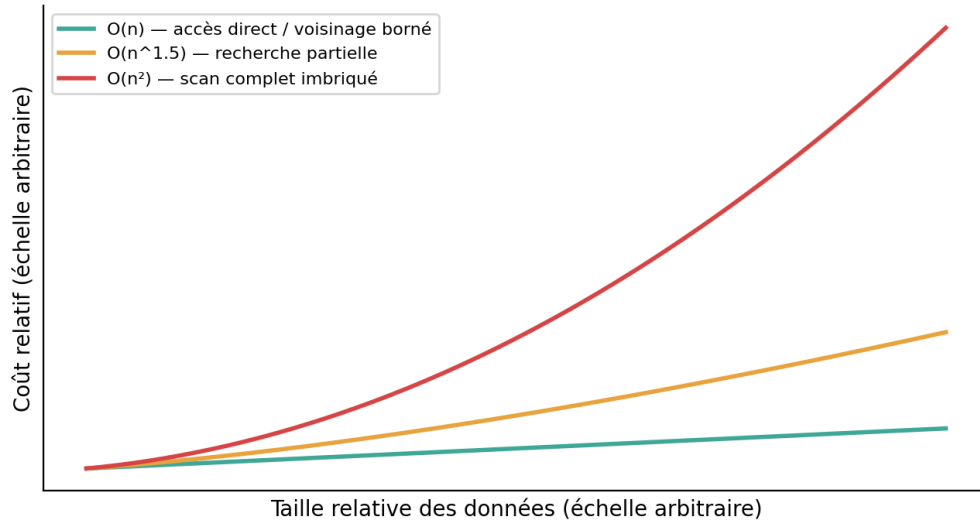


Figure — Rappel pédagogique sur la complexité algorithmique (schéma générique).

3. Profil mémoire — allocations

Type de profil : `alloc_space`. Total : **394.83MB**.

Fonction	Flat	Flat %	Cumulé	Cumulé %
jeu-de-la-vie/game.(*Board).Step	383.41MB	97.11%	383.41MB	97.11%
jeu-de-la-vie/game.RandomBoard	5.52MB	1.40%	6.21MB	1.57%
runtime.mallocgc	2.50MB	0.63%	2.50MB	0.63%
jeu-de-la-vie/game.BenchmarkTick	0	0.00%	390.13MB	98.81%
testing.(*B).launch	0	0.00%	385.48MB	97.63%
testing.(*B).run1.func1	0	0.00%	5.15MB	1.30%
testing.(*B).runN	0	0.00%	390.63MB	98.94%

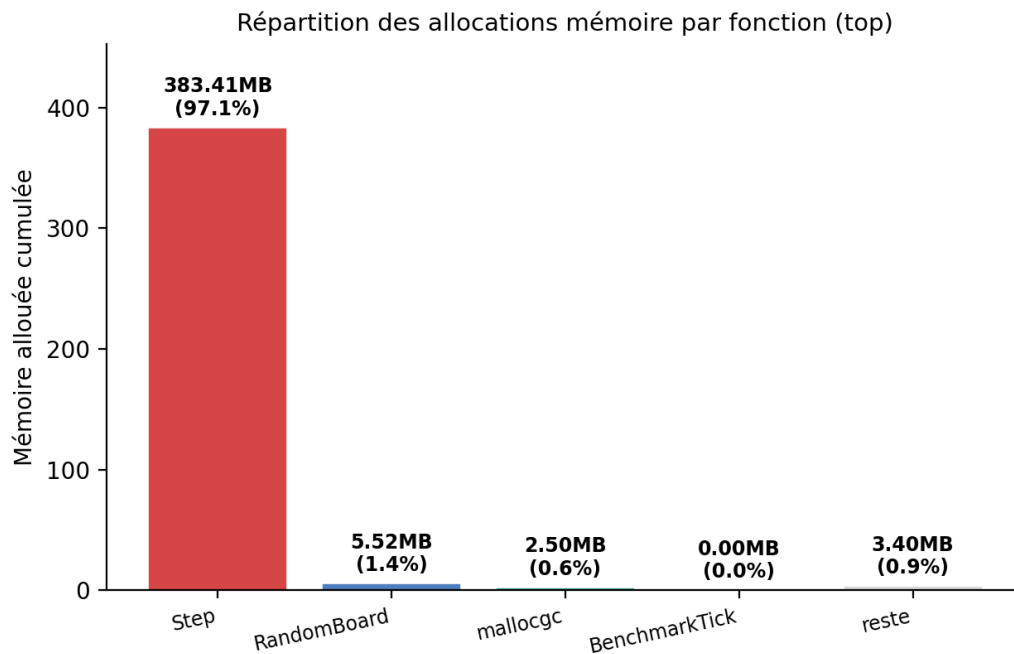


Figure — Répartition des allocations mémoire par fonction.

File: game.test.exe
 Build ID: C:\Users\sadou\JeuDeLaVie\game.test.exe2026-09-23 13:49:53.6122442 +0200 CEST
 Type: alloc_space
 Time: 2026-09-23 14:07:20 CEST
 Showing nodes accounting for 391.43MB, 99.14% of 394.83MB total
 Dropped 41 nodes (cum <= 1.97MB)
 See <https://git.io/JfYMW> for how to read the graph

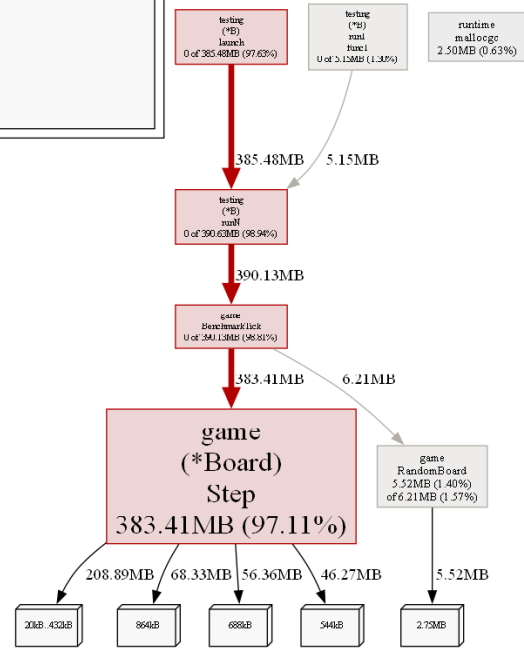


Figure — Graphe d'appels mémoire (go tool pprof -png).

4. Limites de ce benchmark & informations manquantes

Générées automatiquement à partir de la configuration détectée :

- **Peu de runs (1).** Avec moins de 5 exécutions, aucune médiane/écart-type fiable n'est disponible. Recommandé : ≥ 10 runs (-count=10 ou hyperfine --min-runs 10).
- **Aucun warmup.** La première exécution peut être biaisée (cache froid, JIT du GC non stabilisé). Ajouter 1-2 warmups avant la mesure.
- **Pas de comparaison avant/après** intégrée à ce rapport — conservez les PDF successifs pour comparer les hotspots et le temps moyen dans le temps.
- **Un seul point de taille de données.** Si applicable, mesurer plusieurs tailles permettrait de vérifier empiriquement la complexité algorithmique observée en section 2.
- **Pas de trace GC détaillée** (GODEBUG=gctrace=1) — utile si les allocations mémoire (section 3) sont significatives.

Notes de l'outil de benchmark

- Mesure du calcul du tick sur une carte 600×600 .
- La carte et le générateur aléatoire utilisent toujours la seed 42.
- La latence API est mesurée à part dans le frontend, sans rendu canvas.
- Les métriques CPU bas niveau, GC détaillé, I/O, réseau et base de données ne sont pas instrumentées dans ce benchmark.

Rapport généré automatiquement le 2026-09-23 14:20:06 par generate_benchmark_report.py, à partir des fichiers les plus récents du dossier de benchmarks. Analyse réalisée avec go tool pprof.